

使用 Cloud Build 持續部署至 Google Kubernetes Engine (GKE)

來源：<https://codelabs.developers.google.com/codelabs/cloud-builder-gke-continuous-deploy?hl=zh-tw>

步驟數：8

瞭解如何使用 Cloud Build 持續將容器化工作負載部署至 GKE。

目錄

1. [總覽](#)
2. [事前準備](#)
3. [準備環境](#)
4. [建立 GKE 應用程式](#)
5. [自動部署 Git 分支](#)
6. [自動部署 Git 主要分支](#)
7. [自動部署 Git 標記](#)
8. [清除所用資源](#)

步驟 1 · 約 0 分鐘

1. 總覽

在本實驗室中，您將學會如何使用 Cloud Build 設定 GKE 的持續推送軟體更新管道。本實驗室重點說明如何針對不同的 Git 事件觸發 Cloud Build 工作，以及如何在 GKE 中自動發布初期測試版本。

請完成下列步驟：

- 建立 GKE 應用程式
 - 自動部署 Git 分支
 - 自動部署 Git 主要分支
 - 自動部署 Git 標記
-

步驟 2 · 約 0 分鐘

2. 事前準備

如要使用本參考指南，您需要 Google Cloud [專案](#)。您可以建立新專案，或選取已建立的專案：

1. 選取或建立 Google Cloud 專案。

[前往專案選取器頁面](#)

1. 啟用專案的計費功能。

[啟用計費功能](#)

3. 準備環境

1. 建立環境變數，以便在本教學課程中使用：



```
export PROJECT_ID=$(gcloud config get-value project)
export PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --
format='value(projectNumber)')

export ZONE=us-central1-b
export CLUSTER=gke-progression-cluster
export APP_NAME=myapp
```

2. 啟用下列 API：

- Resource Manager
- GKE
- Cloud Source Repositories
- Cloud Build
- Container Registry



```
gcloud services enable \
  clouddresourcemanager.googleapis.com \
  container.googleapis.com \
  sourcerepo.googleapis.com \
  cloudbuild.googleapis.com \
  containerregistry.googleapis.com \
  --async
```

3. 複製範例來源並切換至實驗室目錄：

```
git clone https://github.com/GoogleCloudPlatform/software-delivery-workshop.git gke-
progression

cd gke-progression/labs/gke-progression
rm -rf ../../.git
```

4. 在範例存放區中，將預留位置值替換為 `PROJECT_ID`：在這個步驟中，您會建立目前環境專屬的各種設定檔例項。如要查看範本更新範例，請執行下列指令。

```
cat k8s/deployments/dev/frontend-dev.yaml.tpl
```

執行下列指令，進行變數替換。

```
for template in $(find . -name '*.tpl'); do envsubst '${PROJECT_ID} ${ZONE} ${CLUSTER}
${APP_NAME}' < ${template} > ${template%.*}; done
```

如要查看替換後的檔案範例，請執行下列指令。

```
cat k8s/deployments/dev/frontend-dev.yaml
```

5. 如果您先前未曾在 Cloud Shell 中使用過 Git，請設定要使用的 `user.name` 和 `user.email` 值：

```
git config --global user.email "YOUR_EMAIL_ADDRESS"
git config --global user.name "YOUR_USERNAME"
```

6. 將範例存放區中的程式碼儲存在 Cloud Source Repositories 中：

```
gcloud source repos create gke-progression
git init
git config credential.helper gcloud.sh
git remote add gcp https://source.developers.google.com/p/${PROJECT_ID}/r/gke-progression
git branch -m main
git add . && git commit -m "initial commit"
git push gcp main
```

7. 建立 GKE 叢集。

```
gcloud container clusters create ${CLUSTER} \
  --project=${PROJECT_ID} \
  --zone=${ZONE}
```

8. 授予 Cloud Build 叢集權限。Cloud Build 會將應用程式部署至 GKE 叢集，因此需要相關權限。

```
gcloud projects add-iam-policy-binding ${PROJECT_ID} \
  --member=serviceAccount:${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com \
  --role=roles/container.developer
```

環境已準備就緒！

步驟 4 · 約 0 分鐘

4. 建立 GKE 應用程式

在本節中，您將建構並部署初始正式版應用程式，並在整個教學課程中使用該應用程式。

1. 使用 Cloud Build 建構應用程式：

```
gcloud builds submit --tag gcr.io/$PROJECT_ID/$APP_NAME:1.0.0 src/
```

2. 手動部署至初期測試和正式環境：使用 `kubectl apply` 指令建立正式環境和初期測試環境的部署項目及相關服務。

```
kubectl create ns production
kubectl apply -f k8s/deployments/prod -n production
kubectl apply -f k8s/deployments/canary -n production
kubectl apply -f k8s/services -n production
```

這裡部署的服務會將流量同時導向初期測試和正式環境部署。

3. 查看運作中的 Pod 數量確認前端有四個 Pod 正在運作，其中三個處理正式環境流量，一個處理初期測試版。也就是說，初期測試版的變更只會影響四分之一 (25%) 的使用者。

```
kubectl get pods -n production -l app=$APP_NAME -l role=frontend
```

4. 擷取正式環境服務的外部 IP 位址。

注意：可能要過幾分鐘後，才會看到負載平衡器外部 IP 位址。

```
kubectl get service $APP_NAME -n production
```

負載平衡器傳回 IP 位址後，請繼續下一個步驟

5. 儲存外部 IP，稍後會用到。

```
export PRODUCTION_IP=$(kubectl get -o jsonpath="{.status.loadBalancer.ingress[0].ip}" --
namespace=production services $APP_NAME)
```

6. 檢查應用程式，查看服務的版本輸出內容。應該會顯示「Hello World v1.0」

```
curl http://$PRODUCTION_IP
```

恭喜！您已部署範例應用程式！接下來，請設定觸發條件，持續部署變更。

5. 自動部署 Git 分支

在本節中，您將設定觸發條件，在提交 `main` 以外的分支版本時，執行 Cloud Build 工作。這裡使用的 Cloud Build 檔案會自動為現有或新的分支版本建立命名空間和部署作業，讓開發人員在與主要分支版本整合前預覽程式碼。

1. 設定觸發條件：這個觸發條件的關鍵元件是使用 `branchName` 參數來比對 `main` 和 `invertRegex` 參數 (設為 `true`)，並變更 `branchName` 模式，比對任何不是 `main` 的內容。供您參考，`build/branch-trigger.json` 中應有下列幾行。

```
"branchName": "main",  
"invertRegex": true
```

此外，與這個觸發條件搭配使用的 Cloud Build 檔案最後幾行，會建立以觸發作業的分支命名的命名空間，然後在新命名空間中部署應用程式和服務。供您參考，您可以在 `build/branch-cloudbuild.yaml` 中找到下列幾行

```
kubectl get ns ${BRANCH_NAME} || kubectl create ns ${BRANCH_NAME}  
kubectl apply --namespace ${BRANCH_NAME} --recursive -f k8s/deployments/dev  
kubectl apply --namespace ${BRANCH_NAME} --recursive -f k8s/services
```

瞭解所用的機制後，請使用下列 `gcloud` 指令建立觸發條件。

```
gcloud beta builds triggers create cloud-source-repositories \  
--trigger-config build/branch-trigger.json
```

2. 如要檢查觸發條件，請前往控制台的 [Cloud Build 觸發條件頁面](#)。[前往「觸發條件」](#)
3. 建立新分支：

```
git checkout -b new-feature-1
```

4. 修改程式碼，指出 `v1.1` 編輯 `src/app.py` 並將回應從 `1.0` 改為 `1.1`

```
@app.route('/')  
def hello_world():  
    return 'Hello World v1.1'
```

5. 修訂變更並推送到遠端存放區：

```
git add . && git commit -m "updated" && git push gcp new-feature-1
```

6. 如要查看建構作業進度，請前往控制台的「Cloud Build 記錄」頁面。[前往「建構作業」](#) 建構作業完成後，請繼續下一步
7. 擷取新部署的分支服務外部 IP 位址。

注意：可能要過幾分鐘後，才會看到負載平衡器外部 IP 位址。


```
kubectl get service $APP_NAME -n new-feature-1
```

負載平衡器傳回 IP 位址後，請繼續下一個步驟

8. 儲存外部 IP，稍後會用到。

```
export BRANCH_IP=$(kubectl get -o jsonpath="{.status.loadBalancer.ingress[0].ip}" --  
namespace=new-feature-1 services $APP_NAME)
```

9. 檢查應用程式。檢查服務的版本輸出內容。應該會顯示「Hello World v1.0」

```
curl http://$BRANCH_IP
```

6. 自動部署 Git 主要分支

在將程式碼發布至實際工作環境前，通常會先將程式碼發布至一小部分的實際流量，再將所有流量遷移至新的程式碼集。

在本節中，您將實作程式碼提交至主要分支版本時啟動的觸發條件。觸發程序會部署 Canary 部署作業，將 25% 的所有即時流量導向新修訂版本。

1. 設定主要分支版本的觸發條件：

```
gcloud beta builds triggers create cloud-source-repositories \
  --trigger-config build/main-trigger.json
```

2. 如要查看新觸發條件，請前往控制台的 [Cloud Build 觸發條件頁面](#)。前往「觸發條件」

3. 將分支版本合併至主線，然後推送至遠端存放區：

```
git checkout main
git merge new-feature-1
git push gcp main
```

4. 如要查看建構作業進度，請前往控制台的 [Cloud Build 記錄頁面](#)。前往「建構作業」建構作業完成後，請繼續下一步

5. 查看伺服器的多個回應執行下列指令，並注意大約 25% 的回應會顯示 Hello World v1.1 的新回應

```
while true; do curl -w "\n" http://$PRODUCTION_IP; sleep 1; done
```

準備好繼續時，請按下 `Ctrl+c` 鍵結束迴圈。

7. 自動部署 Git 標記

使用一小部分流量驗證 Canary 部署後，即可將部署作業發布至其餘的即時流量。

在本節中，您將設定在存放區中建立標記時啟動的觸發條件。觸發程序會為映像檔加上適當的標記，然後將更新部署至正式環境，確保 100% 的流量都能存取已加上標記的映像檔。

1. 設定代碼觸發條件：

```
gcloud beta builds triggers create cloud-source-repositories \
  --trigger-config build/tag-trigger.json
```

2. 如要查看新觸發條件，請前往控制台的 [Cloud Build 觸發條件頁面](#)。前往「觸發條件」

3. 建立新標記並推送至遠端存放區：

```
git tag 1.1
git push gcp 1.1
```

4. 如要查看建構作業的進度，請前往控制台的 [Cloud Build 記錄頁面](#)。前往「建構」

5. 查看伺服器傳回的多個回應執行下列指令，並注意 100% 的回應都顯示「Hello World v1.1」這個新回應。在 GKE 中部署新 Pod 並檢查健康狀態時，可能需要一些時間

```
while true; do curl -w "\n" http://$PRODUCTION_IP; sleep 1; done
```

準備好繼續時，請按下 `Ctrl+c` 鍵結束迴圈。恭喜！您在 Cloud Build 中為分支和標記建立 CI/CD 觸發條件，將應用程式部署至 GKE。

8. 清除所用資源

刪除專案

警告：刪除專案會出現以下結果：專案中的所有內容都會刪除。如果您之前使用現有的專案來進行本教學課程，當您刪除該專案時，也會一併刪除您在該專案中完成的所有其他工作。

1. 前往 Cloud Console 中的「[管理資源](#)」頁面。
 2. 在專案清單中選取要刪除的專案，然後點按「刪除」。
 3. 在對話方塊中輸入專案 ID，然後按一下「Shut down」(關閉) 即可刪除專案。
-